

Data Handoff — Ilyas → Dalibor

Project 11_02 — Near Real-Time Financial Forecasting using Amazon SageMaker

Overview

Ilyas's job (steps 1–2) is to fetch financial data and news from external APIs and push them into SQS queues every X minutes.

Dalibor (steps 3–4) consumes the messages from those SQS queues, cleans them with AWS Glue, and stores the result in S3 ready for Amazon SageMaker.

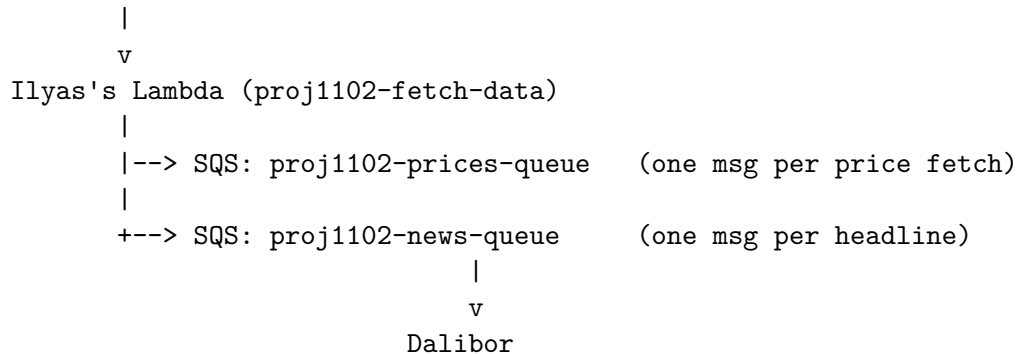
The handoff is essentially: **Ilyas pushes JSON messages, Dalibor pulls them.**

Context: How Ilyas's Lambda is triggered

Amazon EventBridge fires a scheduled rule every X minutes (e.g. every 5 minutes). That rule invokes Ilyas's Lambda `proj1102-fetch-data`, which:

1. Calls Yahoo Finance / Alpha Vantage to get the latest prices for each tracked ticker.
2. Calls NewsAPI (or Alpha Vantage News) to get latest headlines.
3. Sends one message to the prices queue per ticker.
4. Sends one message to the news queue per headline.

EventBridge (every 5 min)



What Ilyas gives Dalibor

1. Two SQS queues

Queue	ARN	Owner	Purpose
proj1102-prices-queue	(Dalibor creates, shares ARN)	Dalibor	Receives price messages
proj1102-news-queue	(Dalibor creates, shares ARN)	Dalibor	Receives news headline messages
proj1102-prices-dlq	(Dalibor creates)	Dalibor	Dead-letter queue for failed price msgs

Queue	ARN	Owner	Purpose
proj1102-news-dlq	(Dalibor creates)	Dalibor	Dead-letter queue for failed news msgs

Dalibor creates the queues (since he owns SQS in step 3) and shares the ARNs with Ilyas, who hardcodes them in his Lambda environment variables.

2. Price message format (proj1102-prices-queue)

One message per ticker per fetch cycle.

```
{
  "ticker": "AAPL",
  "timestamp": "2026-04-18T10:00:00Z",
  "price": 185.42,
  "volume": 1234567,
  "source": "yahoo_finance"
}
```

Field	Type	Required	Example	Notes
ticker	string	yes	AAPL	Symbol from agreed list
timestamp	string	yes	2026-04-18T10:00:00Z	ISO-8601 UTC, no microseconds
price	float	yes	185.42	Latest closing price
volume	integer	yes	1234567	Trading volume
source	string	yes	yahoo_finance	API the data came from (yahoo_finance / alpha_vantage)

3. News message format (proj1102-news-queue)

One message per headline. Each fetch cycle may produce 0 to N news messages.

```
{
  "ticker": "AAPL",
  "timestamp": "2026-04-18T10:00:00Z",
  "headline": "Apple announces record iPhone sales",
  "url": "https://example.com/article",
  "source": "newsapi"
}
```

Field	Type	Required	Example	Notes
ticker	string	yes	AAPL	Which company the headline is about

Field	Type	Required	Example	Notes
<code>timestamp</code>	string	yes	2026-04-18T10:00:00Z	When the headline was fetched (ISO-8601 UTC)
<code>headline</code>	string	yes	Apple announces record iPhone sales	The text Comprehend will analyze
<code>url</code>	string	optional	https://...	Source link (for reference only)
<code>source</code>	string	yes	newsapi	API the headline came from

Frequency and volume

Setting	Agreed value
Fetch frequency	Every 5 minutes (configurable)
Tickers tracked	AAPL, TSLA, BTC-USD, ETH-USD (4 tickers)
Price messages per cycle	4 (one per ticker)
News messages per cycle	0–20 (variable, depends on what’s published)
Daily message volume (rough)	~1,150 prices + ~500 news = ~1,650/day

This stays comfortably inside SQS’s 1M-requests/month free tier.

SQS message attributes (recommended)

To help Dalibor’s Glue job, Ilyas includes a few **SQS message attributes** alongside the JSON body:

Attribute	Type	Example	Purpose
<code>MessageGroupId</code> (FIFO) or <code>MessageDeduplicationId</code>	string	AAPL-2026-04-18T10:00:00Z	Prevents duplicate price msgs if Lambda retries
<code>Source</code>	string	yahoo_finance	Lets Dalibor route by source if needed

If we use **standard SQS** (cheaper, at-least-once), Dalibor’s Glue must dedupe by (`ticker`, `timestamp`). If we use **FIFO SQS**, dedup is automatic. Decision: **standard SQS** is fine for our volume.

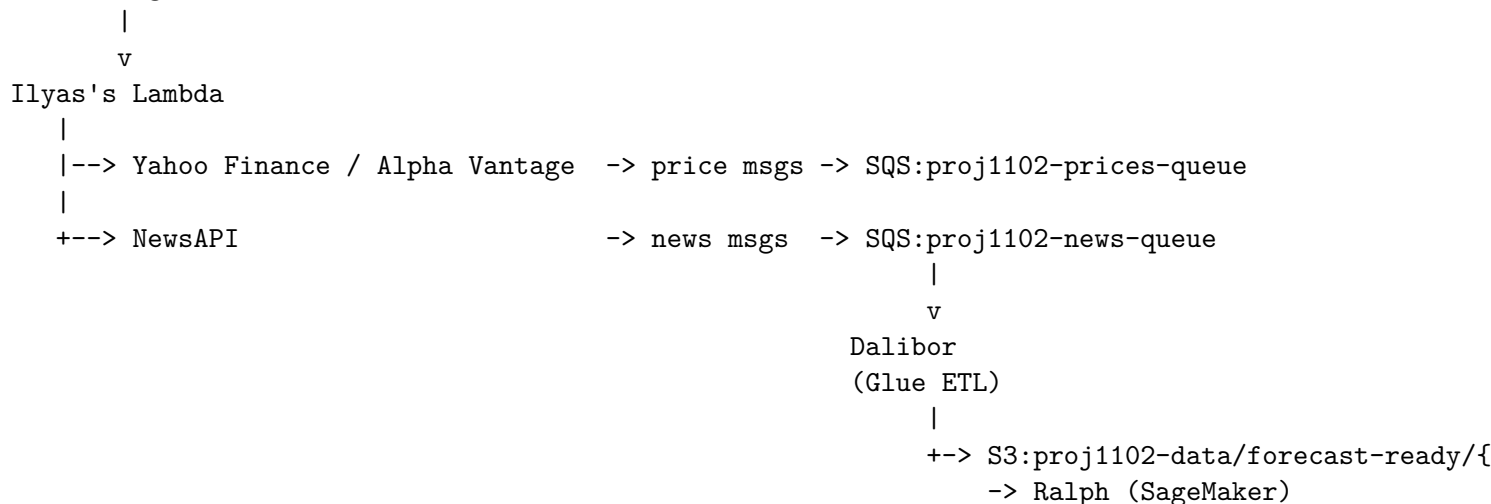
Error handling

- If the external API returns an error, Ilyas’s Lambda logs it to CloudWatch and **does not push a message** for that ticker that cycle. The next cycle (5 min later) tries again.

- If the SQS push itself fails 3 times, the message goes to the DLQ (proj1102-prices-dlq or proj1102-news-dlq).
- Dalibor's Glue job ignores DLQ messages — Arthur's CloudWatch alarm watches the DLQ depth and alerts the team if it grows.

Full data flow (summary)

EventBridge (5 min schedule)



What Dalibor can start building now (with dummy messages)

Dalibor does not need to wait for Ilyas's real Lambda. Once the SQS queues exist, he can manually push test messages with the AWS CLI to develop and test his Glue job.

Dummy price message (push via AWS CLI):

```

aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/<account-id>/proj1102-prices-queue \
  --message-body '{
    "ticker": "AAPL",
    "timestamp": "2026-04-18T10:00:00Z",
    "price": 185.42,
    "volume": 1234567,
    "source": "yahoo_finance"
  }'

```

Dummy news message:

```

aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/<account-id>/proj1102-news-queue \
  --message-body '{
    "ticker": "AAPL",
    "timestamp": "2026-04-18T10:00:00Z",
    "headline": "Apple announces record iPhone sales",
    "url": "https://example.com/article",
  }'

```

```
"source": "newsapi"
}'
```

Dalibor can write a small loop that pushes 50 test messages spread over a fake time range, so his Glue job has something realistic to clean.

Things to freeze between Ilyas and Dalibor (agree before coding)

Item	Agreed value
Prices queue name	<code>proj1102-prices-queue</code>
News queue name	<code>proj1102-news-queue</code>
Queue type	Standard SQS
Region	(agreed AWS region — e.g. <code>us-east-1</code>)
Fetch frequency	Every 5 minutes
Tickers tracked	AAPL, TSLA, BTC-USD, ETH-USD
Timestamp format	ISO-8601 UTC (e.g. <code>2026-04-18T10:00:00Z</code>)
Price JSON keys	<code>ticker</code> , <code>timestamp</code> , <code>price</code> , <code>volume</code> , <code>source</code>
News JSON keys	<code>ticker</code> , <code>timestamp</code> , <code>headline</code> , <code>url</code> , <code>source</code>
DLQ names	<code>proj1102-prices-dlq</code> , <code>proj1102-news-dlq</code>

Any change to the queue names or JSON keys must be announced to the group — both Ilyas's writer and Dalibor's reader will break otherwise.

Quick checklist

Ilyas's side

- ☐ EventBridge rule `proj1102-scheduler` created (every 5 min)
- ☐ Lambda `proj1102-fetch-data` deployed
- ☐ Lambda has API keys (Yahoo Finance, Alpha Vantage, NewsAPI) in env vars or Secrets Manager
- ☐ Lambda receives queue ARNs from Dalibor and writes them in env vars
- ☐ JSON output matches the schemas above exactly
- ☐ Error handling: failed API call logged to CloudWatch, no message pushed
- ☐ Lambda ARN + log group shared with Arthur (for Step Functions / CloudWatch)
- ☐ Tested with at least one ticker producing real messages

Dalibor's side

- ☐ SQS queues `proj1102-prices-queue` and `proj1102-news-queue` created
- ☐ DLQs created and attached (max 3 retries)
- ☐ Queue ARNs shared with Ilyas
- ☐ Glue job reads from both queues (or from S3 if buffering raw msgs there first)
- ☐ Glue dedupes on (`ticker`, `timestamp`)
- ☐ Tested with dummy messages from AWS CLI